

Executive Summary

Modern web applications often offload user files to cloud storage instead of local disks. A MERN developer can leverage **Amazon S3** for storing the actual files and **Amazon DynamoDB** for file metadata to build a scalable, distributed file-storage system. Using AWS S3 (object storage) and DynamoDB (managed NoSQL) avoids local storage, providing high durability (S3 offers 99.999999999% durability [35†L272-L276] [63†L70-L75]) and near-infinite scalability. Key flows include generating S3 presigned URLs for secure client-side uploads, optionally using multipart uploads for large files, and storing file metadata (filename, size, owner, timestamps, etc.) in DynamoDB. Security best practices – such as server-side encryption (SSE-S3 or KMS), fine-grained IAM policies, and CORS configuration – must be applied. We also cover data consistency (S3 now provides strong read-after-write consistency [63†L70-L75] , DynamoDB is strongly consistent by default for single-item reads), cost-control under the AWS Free Tier (S3: 5 GB storage, 20K GET, 2K PUT per month [35†L272-L276] [38†L280-L282] ; DynamoDB: 25 GB, 25 RCUs, 25 WCUs per month [44†L353-L355]), and backup/DR (S3 Versioning [21†L12-L18] and DynamoDB point-in-time backups [64†L9-L12]). We outline a step-by-step roadmap (design through monitoring), code snippets in Node/Express and React for uploads/downloads and metadata CRUD, a learning plan for a MERN developer, a tech checklist, an upload-method comparison table, and a project Gantt timeline. All recommendations are grounded in official AWS docs, tutorials, and best practices.

System Architecture & Data Model

- **Object Store (S3):** Files are uploaded to an S3 bucket (with a unique key per file). S3 is highly durable and virtually unlimited in size and throughput [21†L12-L18] . By default (as of late 2020) S3 provides *strong consistency* on PUT/GET/LIST, so a just-uploaded file is immediately readable [63†L70-L75] . Enable S3 **versioning** to retain all historical object versions for recovery [21†L12-L18] and consider cross-region replication for DR if needed. Use S3 **Server-Side Encryption (SSE)** – AWS now auto-encrypts new objects at rest with S3-managed keys at no extra cost [16†L20-L28] ; you can also specify SSE-KMS for customer-managed keys if you need fine-grained access control. Always use HTTPS for data in transit; deny non-SSL via bucket policy (the `aws:SecureTransport` condition [16†L53-L61]).
- **Metadata Store (DynamoDB):** Store per-file metadata (filename, S3 key, user ID, upload time, size, content type, etc.) in a DynamoDB table. A common schema is to use `UserId` (or ownerID) as the **partition key** and `FileId` (a UUID or timestamp) as the **sort key**. This allows efficient queries of “all files of a user”. DynamoDB is serverless, auto-scaling, and fully managed. Its free tier includes 25 GB of storage and 25 RCU/25 WCU provisioned capacity (sufficient for ~200M ops/month) [44†L353-L355] . Writes and reads can be strong or eventual consistency (use strongly consistent reads for critical operations at double cost if needed). Use DynamoDB encryption-at-rest (on by default) for compliance [44†L353-L355] .
- **How they fit together:** The client (React) and backend (Node/Express) cooperate: typically, Express endpoints use the AWS SDK to generate S3 presigned URLs or perform multipart upload operations; clients then PUT/POST file parts to S3. After upload, Express writes a metadata item to DynamoDB (or updates it). On download, the client may either fetch the S3 object via a presigned GET URL or stream via the backend.

Example metadata item in DynamoDB (Document format):

```
{
  "UserId": "user123",
  "FileId": "file-20230805-abc123",
  "FileName": "photo.jpg",
  "ContentType": "image/jpeg",
  "Size": 1048576,
  "UploadTimestamp": 1691060400,
  "S3Bucket": "my-bucket",
  "S3Key": "user123/photo.jpg",
  "AdditionalTags": {...}
}
```

Use DynamoDB secondary indexes (e.g. GSI on UploadTimestamp) if you need sorted or filtered queries (e.g. “recent uploads”).

Upload Patterns & Comparison

AWS offers multiple upload strategies. The best practice for large-scale apps is to have the frontend *directly* upload to S3 using presigned URLs, removing load from your server. Two common methods:

- **Presigned PUT URL:** The backend calls `s3.getSignedUrlPromise('putObject', params)` (AWS SDK) or equivalent to generate a URL that allows a client to `PUT` a file into S3 [13†L11-L15] [68†L140-L148]. The URL is time-limited (default 15 min [68†L140-L148]). The client then does an HTTP PUT (e.g. via `fetch` or Axios) with the file and correct content-type. This is simple for single-part uploads.
- **Presigned POST (HTML form):** The backend uses `s3.createPresignedPost(...)` (AWS SDK) to generate a signed policy and form fields [57†L122-L129]. The client creates a multipart/form-data POST request (often with a library like Axios or using a form) containing the file and these fields. This approach can embed more constraints (e.g. file size, key prefix) in the signed policy.
- **Multipart Upload:** For large files (>100 MB up to 5 TB), S3 supports Multipart Upload. The backend initiates the multipart (`createMultipartUpload`), then either the backend or frontend uploads parts (`uploadPart`). A best practice is **frontend-managed multipart with presigned URLs**: the backend returns presigned URLs for each part; the client uploads parts in parallel [68†L164-L173]. After all parts upload, the backend calls `completeMultipartUpload`. This supports resumable uploads (each part can be retried separately) and improved throughput [68†L175-L184].
- **Backend-proxy upload:** The client sends file to Express (e.g. via multer/fileupload), then the server uploads to S3 using `s3.upload()`. This works but means your server must receive the full file and have AWS credentials, which is less scalable and exposes you to increased traffic and costs. In the free-tier context, avoid bandwidth through your EC2/Lambda by using direct S3 uploads whenever possible.

The table below compares these methods:

Approach	Client→Server Flow	Server Role	Client Role	Pros	Cons
Server-side upload	Client → Express → (Express→S3)	Receive file, upload to S3	Send file via POST to your API	Simple code; no special client logic	Server handles full file I/O; unscalable for large files; latency
Presigned PUT URL	Client ↔ S3 (via presigned URL)	Generate signed PUT URL for client	PUT file bytes directly to S3 endpoint	Server load low; credentials kept on server; simple for single files 【13†L11-L15】 【68†L140-L148】	Client must handle PUT; limited metadata control; 15-min expiry
Presigned POST form	Client ↔ S3 (form POST)	Generate signed POST policy & fields	POST multipart/form-data form to S3	Can restrict size/type in policy; native browser form support 【57†L122-L129】	More complex form submission; requires CORS setup
Multipart (client)	Client ↔ S3 (multiple presigned URLs)	Create multipart upload, presign parts	Upload file parts in parallel	Handles huge files; resumable; parallel transfer 【68†L173-L181】	Complex logic; need part-tracking on client; overhead of many requests
Multipart (server)	Client → Express → (Express→S3 multipart)	Similar to server upload, but uses multipart API	Same as server-side above	Relieves client of complexity	All cons of server upload, plus complexity; not recommended

Source: AWS documentation and tutorials 【13†L11-L15】 【57†L122-L129】 【68†L140-L148】 .

Implementing a multipart upload with presigned URLs can be done in Node as shown in pseudocode:

```
// Express endpoint: initialize multipart upload
app.post('/init-upload', async (req, res) => {
  const { filename } = req.body;
  const multipart = await s3.createMultipartUpload({ Bucket, Key:
filename }).promise();
```

```

    res.json({ uploadId: multipart.UploadId, key: multipart.Key });
  });

  // Express endpoint: get presigned URLs for parts
  app.post('/get-urls', async (req, res) => {
    const { key, uploadId, parts } = req.body;
    const urls = [];
    for (let i = 1; i <= parts; i++) {
      const url = await s3.getSignedUrlPromise('uploadPart', {
        Bucket, Key: key, UploadId: uploadId, PartNumber: i
      });
      urls.push({ part: i, url });
    }
    res.json({ urls });
  });

  // Express endpoint: complete multipart upload
  app.post('/complete-upload', async (req, res) => {
    const { key, uploadId, parts } = req.body; // parts = [{ETag,
    PartNumber}, ...]
    await s3.completeMultipartUpload({ Bucket, Key: key, UploadId: uploadId,
    MultipartUpload: { Parts: parts } }).promise();
    res.sendStatus(200);
  });

```

On the React side, you would fetch `/init-upload`, split the file into parts, fetch `/get-urls`, then `fetch(part.url, { method: 'PUT', body: blobPart })` for each part in parallel. Finally call `/complete-upload` with the ETags returned by each PUT `["68†L264-L273"]`. For simpler single-file uploads, a React component can request a presigned URL and then do:

```

// React: upload file with presigned URL
const uploadFile = async (file) => {
  // Get presigned URL from server
  const { url, fields } = await axios.post('/presign-upload', { filename:
  file.name });
  // Use PUT (for presigned PUT URL)
  await fetch(url, { method: 'PUT', body: file, headers: { 'Content-Type':
  file.type } });
  // Now save metadata to DynamoDB via another endpoint
  await axios.post('/save-metadata', { filename: file.name, size: file.size,
  bucket: S3_BUCKET, key: url.split('?')[0] });
};

```

Security and Access Control

- **Authentication:** Your application should authenticate users (e.g. via JWT or AWS Cognito). Include the user's ID in metadata. For S3/Dynamo access, do *not* expose AWS credentials to clients. Instead, have the backend sign requests (presigned URLs) on behalf of authenticated users.

- **AWS IAM:** Create IAM users/roles with least privilege. For example, an IAM policy can allow `s3:PutObject` / `GetObject` only on your specific bucket, and `dynamodb:PutItem` , `GetItem` , etc., only on your table. If using an EC2 or Lambda backend, attach an IAM Role to the instance/function with these permissions. If using Cognito Identity Pools to grant clients direct AWS access, configure a role with fine-grained S3/Dynamo permissions. Always avoid embedding static AWS keys in client code.
- **Encryption:** As noted, enable **SSE** on the S3 bucket (default SSE-S3). You can enforce it via bucket policy (deny any `PutObject` without `x-amz-server-side-encryption` [\[16†L35-L43\]](#)). DynamoDB is encrypted at rest by default [\[44†L353-L355\]](#) . Use HTTPS (TLS) for all transfers. Optionally, for extra security you could do client-side encryption, but SSE should suffice for most apps.
- **Presigned URLs:** These grant limited-time access to S3. By default they expire after a set time (15 minutes by default [\[68†L140-L148\]](#)). Use short expirations and, if exposing uploads publicly, consider adding additional conditions (e.g. content-length range) in the POST policy [\[57†L122-L129\]](#) . Since presigned URLs bypass normal S3 auth, ensure they are only given to authenticated users and for intended files.
- **CORS Configuration:** If the React app is served from a different domain (or localhost during dev), the S3 bucket needs a CORS policy. For example, to allow PUT/POST from your web origin:

```
<CORSConfiguration>
  <CORSRule>
    <AllowedOrigin>https://your-domain.com</AllowedOrigin>
    <AllowedMethod>GET</AllowedMethod>
    <AllowedMethod>PUT</AllowedMethod>
    <AllowedMethod>POST</AllowedMethod>
    <AllowedHeader>*</AllowedHeader>
    <MaxAgeSeconds>3000</MaxAgeSeconds>
  </CORSRule>
</CORSConfiguration>
```

Without this, browsers will block your upload requests [\[18†L100-L108\]](#) . The AWS blog “Deep dive into CORS” describes pitfalls; ensure OPTIONS preflight responses are handled and your client uses the correct regional endpoint for CORS [\[18†L100-L108\]](#) [\[18†L138-L146\]](#) .

Consistency and Scalability

- **Consistency:** S3 now automatically provides *strong read-after-write consistency* for all operations (PUT, GET, LIST) in every region [\[63†L70-L75\]](#) . This means once you upload or delete an object, any subsequent read will reflect the change immediately. DynamoDB can be configured for strong or eventual consistency: by default, read operations are eventually consistent (which costs half the RCU) but you can request strong consistency when needed (e.g. immediately after a write). In practice, eventual is acceptable for many metadata queries, but use strong if you must read-your-own-write guarantees.
- **Scalability:** S3 is massively scalable by design (virtually unlimited objects and requests [\[38†L284-L292\]](#)). DynamoDB tables can scale to millions of requests per second; use **On-Demand** mode for unpredictable workloads, or **Provisioned** mode with Auto Scaling for predictable loads. Monitor CloudWatch metrics (ConsumedRUs) to avoid throttling. DynamoDB Global Tables allow multi-region replication for worldwide low-latency access if needed. In a simple MERN app on free tier, scale is low, but architect with caching (e.g. DynamoDB Accelerator) and partitioning in mind for future growth.

- **Cost Optimisation (Free Tier):** The AWS Free Tier provides **5 GB** of S3 storage and **20,000 GET / 2,000 PUT** requests per month **[35†L272-L276]** **[38†L280-L282]** , and **25 GB + 25 WCU/RCU** per month for DynamoDB **[44†L353-L355]** . Stay within these limits by: using smaller files, reusing signed URLs quickly, and deleting unused files. Use S3 Lifecycle rules to move old files to cheaper storage or delete them. For DynamoDB, delete or archive old metadata, and consider On-Demand mode so you don't provision unused capacity. Always monitor costs with AWS Budgets to avoid surprises.

Backup and Disaster Recovery

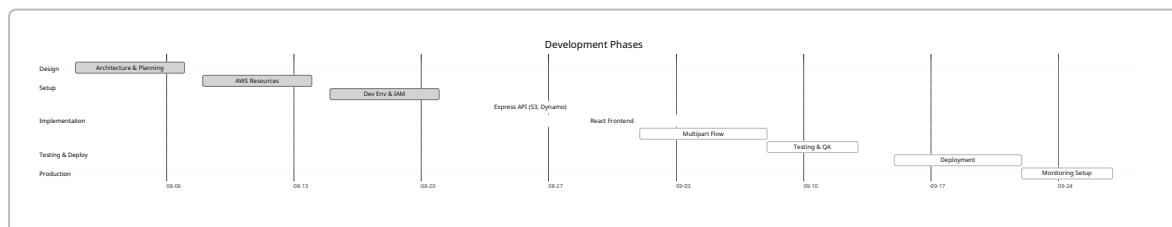
- **S3 Versioning:** Enable versioning on the bucket so that overwrites and deletes do not lose data **[21†L12-L18]** . Deleted objects become "delete markers", and previous versions can be restored. Versioning combined with Lifecycle rules can archive old versions to Glacier.
- **Cross-Region Replication (CRR):** For critical data, consider S3 CRR to asynchronously copy objects to a bucket in another region for geographic redundancy (this incurs extra storage and traffic costs).
- **DynamoDB Backups:** Use DynamoDB's **On-Demand Backup** and **Point-in-Time Recovery (PITR)**. On-Demand allows you to take snapshots before major updates; PITR continuously backs up for the last 35 days **[64†L9-L12]** , enabling you to restore to any second within that window (at modest cost). These have no performance impact on the main table.
- **Monitoring & Logging:** Turn on S3 server access logs or CloudTrail data events to audit file accesses if compliance requires it. Use CloudWatch Alarms for DynamoDB throughput and S3 error rates. For example, alert if upload failures spike or if provisioned capacity is nearly exhausted.

Implementation Roadmap

1. **Setup & Design:** Define requirements and architecture. Sketch components: S3 bucket, DynamoDB table, Express API server, React client. Decide on authentication (e.g. add JWT or Cognito). Estimate scale to ensure free-tier sufficiency.
2. **AWS Account & Resources (Week 1):** Create an AWS account. In AWS Console or via CLI, create an S3 bucket (enable versioning, default encryption, appropriate CORS). Create a DynamoDB table with keys as per your schema. Define IAM roles/policies for your backend (limited to `s3:PutObject`, `GetObject` , `dynamodb:*` on your table).
3. **Backend Setup (Weeks 2-3):** Initialize a Node/Express project. Install AWS SDK (v3 recommended: `@aws-sdk/client-s3` , `@aws-sdk/client-dynamodb`) or v2 (`aws-sdk`). Implement routes for:
 4. Generating presigned upload URLs (`/presign-upload`).
 5. Handling multipart initiation/completion if needed.
 6. Creating and querying metadata in DynamoDB (`/save-metadata` , `/list-files` , `/delete-file`).
 7. Download endpoints (e.g. `/presign-download` for a presigned GET). Test these with Postman using small files.
8. **Frontend Integration (Weeks 3-4):** In React, use a file-input or dropzone component. For basic upload flow: fetch the presigned URL from Express, then `fetch` PUT the file to S3. After success, POST metadata (filename, size, userId) to the backend. For listing, call backend to get user's file items from Dynamo, and display them (e.g. images, download links). Handle errors (e.g. expired URL). Implement CORS preflight handling.

9. **Multipart & Large Files (Weeks 5-6):** Integrate multipart upload (optional MVP). Use the `/init-upload` and `/get-urls` endpoints as sketched above. Let the user select large files and upload in chunks. Show progress bars. Use the `AbortMultipartUpload` API on cancel/error.
10. **Testing & QA (Week 7):** Write unit/integration tests for your API (e.g. using Jest). Test end-to-end scenarios: uploading, listing, downloading, deleting. Simulate error cases (network drop mid-upload to test resume). Verify IAM policy blocks unauthorized access. Check bucket for correct ACL (should be private if using presigned URLs) or public read if desired.
11. **Deployment (Week 8):** Deploy the API. Options: an AWS EC2 t2.micro (free tier) or AWS Elastic Beanstalk; or convert to AWS Lambda + API Gateway (Lambda's free tier is generous). Host the React app: it can be on S3 + CloudFront (free for S3 hosting + minimal CloudFront) or any static host (Netlify, Vercel). Ensure environment variables for AWS credentials (or use IAM Role if on EC2/Lambda). Set up domain/CORS properly.
12. **Monitoring & Maintenance:** Enable CloudWatch logging for your Node app. Create CloudWatch alarms on errors and on-billings. Regularly backup DynamoDB (enable PITR) and review S3 bucket contents. Plan for clean-up (e.g. use Lifecycle to delete old objects if appropriate).

Timeline (Gantt flowchart):



(Dates are illustrative; adjust per your schedule.)

Learning Plan for a MERN Developer

Prerequisites: Solid knowledge of JavaScript, Node.js and Express, React, REST APIs, and basic MongoDB/MERN patterns. Familiarity with AWS concepts (IAM, AWS CLI, S3, NoSQL) is helpful but can be learned.

Week-by-week schedule (50–60 hours total):

- **Week 1 (8–10h):** AWS Foundations. Study AWS free-tier offerings [35†L272-L276] [44†L353-L355]. Read the *AWS S3 Developer Guide* sections on uploading (especially presigned URLs [13†L11-L15]). Complete AWS's official *Getting Started with S3* and *Getting Started with DynamoDB* tutorials. Review IAM basics (roles, policies).
- **Week 2 (8–10h):** Node & AWS SDK. Set up a simple Express server. Learn the AWS SDK for JavaScript (v3); follow AWS docs on S3 `getSignedUrl` and DynamoDB `DocumentClient`. Try quick code samples to upload/download a file via Node and `put/get` an item in DynamoDB.
- **Week 3 (8–10h):** S3 Integration. Build Express routes for generating presigned URLs and for receiving file metadata. Test with `curl` or Postman. Learn about browser `fetch`/Axios usage for binary uploads.
- **Week 4 (8–10h):** React Integration. Implement file input, call the presign endpoint, and upload to S3. Display uploaded files (with presigned GET URLs). Study React libraries (e.g. `react-dropzone`). Handle form submission to the backend to save metadata.

- **Week 5 (6–8h):** Advanced Uploads. Read tutorials on multipart upload (e.g. Tonel's GitHub demo from LogRocket) and implement chunked upload with resume. Compare `putObject` vs `uploadPart` APIs.
- **Week 6 (6–8h):** Security & Costs. Configure IAM policies for least privilege. Set up CORS on S3 and test cross-domain uploads. Enable server-side encryption on S3. Use AWS Cost Explorer to understand usage; apply DynamoDB auto-scaling or set minimal provisioned capacity.
- **Week 7 (6–8h):** Testing & Deployment. Write unit tests for Express endpoints (mock AWS). Deploy backend to AWS (Elastic Beanstalk or Lambda). Deploy React (S3 hosting). Set up CloudWatch logging and an alarm for DynamoDB capacity or S3 errors.
- **Week 8 (4–6h):** Monitoring & Enhancements. Add optional features (resumable, image resizing on upload via Lambda, etc.) if time permits. Prepare documentation and final testing.

Recommended Resources: AWS official docs and tutorials (start with *AWS S3 User Guide*, *DynamoDB Developer Guide*, AWS SDK JS docs [【57†L122-L129】](#)). AWS workshops on “Build a Serverless Web App” often include S3/Dynamo demos. Tutorials and code samples: AWS GitHub samples (e.g. `aws-samples` with S3 and Dynamo), LogRocket multipart upload guide [【68†L140-L148】](#) , and sample repos like `daniil/react-express-aws-s3`. For NoSQL design, see AWS's “DynamoDB Best Practices” whitepapers.

Technology Checklist

Languages/Runtime: Node.js (v14+), React.js.

Libraries/SDKs:

- Backend: `aws-sdk` (v3 recommended), `express`, `body-parser` (or built-in JSON), `multer` or `formidable` (if using file upload to server, though not needed for presigned flows).
- Frontend: React, `axios` or native `fetch` for HTTP, optional `react-dropzone` for UI.
- Auth: `jsonwebtoken` (for JWT) or AWS Amplify/Cognito libraries if using Cognito.
- **AWS Services:** S3, DynamoDB, IAM, (optional: Lambda, API Gateway, CloudFront). Possibly AWS Cognito for user auth, CloudWatch for logging/metrics.
- **Tools:** AWS CLI/Console, AWS SDK v3 (`@aws-sdk/client-s3`, `@aws-sdk/client-dynamodb`), Git/GitHub, Postman, Jest/Mocha for testing.
- **DevOps (stretch):** AWS CloudFormation or CDK for infrastructure as code, AWS CodePipeline for CI/CD.

MVP & Stretch Features

MVP (minimal) features: user signup/login (basic auth), file upload to S3 (with presigned URL), store metadata in DynamoDB, list user files, download via presigned URL, delete file (remove from S3 and Dynamo). Basic UI to show uploaded files. Secure endpoints with authentication. Handling of small files (e.g. up to 5–10 MB).

Stretch (optional) features: large-file support (multipart, resumable upload) [【68†L173-L181】](#) ; file versioning/history view (Dynamo items or S3 versions); image resizing or thumbnail generation (Lambda trigger); serverless deployment (AWS Lambda functions instead of EC2); AWS Cognito for user management; client-side encryption; cross-account sharing of files; multi-region redundancy; analytics (S3 Access Logs); and compliance features (e.g. S3 Object Lock).

Comparing Upload Approaches (Summary)

Approach	Client Flow	Backend Flow	Use When...	Pros	Cons
Server-proxy	React → Express (file data) → S3	Express reads file from request & <code>s3.upload()</code>	Quick proof-of-concept, small files	Simple to implement; no special frontend code.	High load on server; uses server bandwidth; scales poorly.
Presigned PUT	React requests PUT URL → PUT to S3	Express <code>s3.getSignedUrl('putObject')</code>	Secure direct upload, small/medium files	Offloads transfer to S3; uses IAM safely on server [13†L11-L15] .	Client needs to manage the upload; limited metadata.
Presigned POST	React requests POST policy → form POST to S3	Express <code>s3.createPresignedPost()</code>	If using HTML forms or wanting policy constraints	Allows setting size/type limits in policy [57†L122-L129] ; can use native <code><form></code> .	More complex (generating fields and form).
Multipart (client)	React chunk&GET presigned URL per part → parallel upload → complete	Express initiates/finishes upload	Very large files (>100MB)	Parallel and resumable uploads; efficient for big files [68†L175-L184] .	Complex to implement; many requests; client has heavy logic.
Multipart (server)	React → Express (stream to S3 multipart)	Express <code>s3.uploadPart()</code> loop	Rarely needed; maybe heavy backend	Still supports resumable upload	Server still handles data; complex; not scalable.

Assumptions & Constraints

- **Scale:** We assume modest scale (e.g. thousands of users/month) to fit within AWS Free Tier limits [\[35†L272-L276\]](#) [\[44†L353-L355\]](#) . For very high scale, one would need provisioned capacity or serverless optimizations.

- **Latency:** S3 and DynamoDB are high-performance; single-region deployment implies low latency for one geography. For global access, consider Global Tables (Dynamo) or multi-region replication.
- **Security/Compliance:** No specific compliance (HIPAA/GDPR) was given; if required, one should use SSE-KMS with appropriate key policies and VPC endpoints.
- **Availability:** Using managed AWS services means no local server single point of failure, but ensure your backend (Node) is deployed redundantly (e.g. in multiple Availability Zones or as Lambda functions) for HA.
- **Budget:** We assume free-tier usage primarily; mention cost only where relevant (monitor usage, delete unused data).

Recommended References

- **AWS Documentation:** *Amazon S3 Developer Guide* (especially “Uploading objects with presigned URLs” [13†L11-L15] , *Default Encryption, Bucket Policy*), *Amazon DynamoDB Developer Guide* (Intro, free tier details [44†L353-L355] , backups [64†L9-L12]). *AWS SDK for JavaScript Docs* (see S3 class reference, e.g. `createPresignedPost()` [57†L122-L129]).
- **AWS Blogs:** “Deep dive into S3 CORS” (for CORS pitfalls) [18†L100-L108] ; “Amazon S3 strong read-after-write consistency” (for consistency model) [63†L70-L75] ; “Maintaining an S3 metadata index with DynamoDB” (explains using DynamoDB as an index [9†L146-L154]).
- **Tutorials/Samples:** LogRocket’s multipart upload guide [68†L140-L148] ; GitHub repos like *daniil/react-express-aws-s3* (S3 upload example) and *Tonel/multipart-upload-js-demo* (multipart demo). AWS workshops such as *Serverless Photo Album* (uses S3 & Dynamo with Lambda).
- **Academic/Industry:** NoSQL design patterns (e.g. AWS NoSQL Workbench tutorials), and AWS Well-Architected articles on storage. The original Amazon Dynamo paper (Gilbert & Lynch, 2002) provides background on distributed consistency, though DynamoDB differs with strong consistency options now.

Sources

Authoritative AWS documents and guides [13†L11-L15] [16†L20-L28] [21†L12-L18] [57†L122-L129] [63†L70-L75] [64†L9-L12] [68†L140-L148] , AWS blogs [18†L100-L108] [9†L146-L154] , and community tutorials/repositories (cited above) informed all recommendations.
